

# Avaliação Prática (Parte 1)

## Pipeline de Extração e Limpeza de Dados (ETL)

*Ciência de Dados*  
Formato: Grupos ou Individual

---

### 1 Objetivo da Avaliação

O objetivo desta primeira etapa é construir o motor de ingestão e tratamento de dados de um pipeline de ETL. Vocês deverão buscar dados reais no **Kaggle** (utilizando as fontes recomendadas abaixo) e tratá-los utilizando boas práticas de Programação Orientada a Objetos (POO) e o **Princípio da Responsabilidade Única (SRP)**, garantindo que o dado final esteja limpo, tipado e unificado.

### 2 Escolha do Tema e Fontes de Dados (Kaggle)

Cada equipe deverá escolher **um** dos temas abaixo. O processo de **Extração (*Extract*)** deve obrigatoriamente consumir as fontes indicadas do Kaggle.

#### Opção A: Logística, Transportes e Segurança Viária

- **Contexto:** Mapear o perfil dos acidentes nas rodovias federais para criar rotas mais seguras e prever custos de seguro de frotas.
- **Fonte Principal (Kaggle):**  
<https://www.kaggle.com/datasets/rafaelborgesgraunke/traffic-accidents-brazil-pt-br>
- **Fonte Secundária (Kaggle):**  
<https://www.kaggle.com/datasets/liamarguedas/brazil-total-highway-crashes-2010-2023>

#### Opção B: Finanças, Mercado e E-Commerce

- **Contexto:** Cruzar o comportamento de compras, meios de pagamento e categorias para entender a saúde do comércio eletrônico no ecossistema brasileiro.
- **Fonte de Dados (Kaggle):**  
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>  
*Nota: Este repositório contém múltiplas tabelas relacionais que devem ser integradas.*

## Opção C: Meio Ambiente e Emergências Climáticas

- **Contexto:** Analisar o histórico de queimadas por estado brasileiro para identificar padrões de degradação ao longo do tempo.
- **Fonte de Dados (Kaggle):**  
<https://www.kaggle.com/datasets/gustavomodelli/forest-fires-in-brazil>

## Opção D: Saúde Pública (Dados do SUS e Leitos)

- **Contexto:** Cruzar dados de infraestrutura e capacidade de atendimento para identificar vulnerabilidades regionais na rede de saúde pública.
- **Fonte de Dados (Kaggle):**  
<https://www.kaggle.com/datasets/gpreda/covid19-vaccination-track-everywhere-in-the-world>  
*Nota: Filtrar os registros correspondentes ao Brasil no processo de extração.*

## Opção E: Educação e Desempenho Acadêmico

- **Contexto:** Analisar fatores socioeconômicos e correlacioná-los com o desempenho e a avaliação final dos estudantes.
- **Fonte de Dados (Kaggle):**  
<https://www.kaggle.com/datasets/cansa23/higher-education-students-performance-evaluation>

# 3 Detalhamento das Atividades Solicitadas

O pipeline deve ser desenvolvido em camadas isoladas utilizando Programação Orientada a Objetos (POO).

## 3.1 Camada de Extração (*Extract*)

Vocês devem criar uma classe especializada exclusivamente na ingestão dos dados.

- **Requisito 1 (Ingestão Programática):** O pipeline deve ser capaz de ler os arquivos baixados do Kaggle (.csv). Como bônus técnico, a equipe pode utilizar a biblioteca oficial `KaggleApi` para baixar os arquivos direto por código.
- **Requisito 2 (Resiliência):** Implementar tratamento de exceções robusto (`try/except`). O código deve prever falhas de arquivos corrompidos, caminhos errados ou colunas ausentes, encerrando o programa sem gerar um *crash* no sistema.
- **Requisito 3 (Logs):** Criar uma classe auxiliar de Log que exiba no terminal o progresso da operação (Ex: [INFO] Iniciando extração do arquivo... [SUCCESS] Processo concluído.).

## 3.2 Camada de Limpeza e Transformação (*Transform*)

Os dados brutos do Kaggle possuem anomalias propositais. Esta classe deve purificá-los.

- **Requisito 1 (Padronização e Codificação):** Tratar erros de codificação de caracteres (acentuações quebradas). Corrigir textos com espaços desnecessários e padronizar strings para letras maiúsculas ou minúsculas.
- **Requisito 2 (Normalização de Datas e Tipos):** Unificar todas as colunas de data para o padrão ISO YYYY-MM-DD. Colunas numéricas e de coordenadas devem ser convertidas estritamente para valores numéricos (`float` / `int`).
- **Requisito 3 (Tratamento de Nulos e Outliers):** Decidir estrategicamente o que fazer com campos críticos nulos (descarte ou preenchimento com valor padrão coerente). A decisão deve ser documentada no código.
- **Requisito 4 (Cruzamento/Merge):** Realizar o `join/merge` das tabelas através de suas chaves comuns (Ex: `order_id`, `id.municipio`, etc.).

## 4 Estrutura do Projeto e Entregáveis

O projeto deve ser versionado no **Git** e seguir rigorosamente a estrutura arquitetural gerada abaixo:

```
meu-projeto-etl/
├── src/
│   ├── extract/
│   │   └── extractor.py           Classe de leitura dos dados
│   ├── transform/
│   │   └── cleaner.py           Regras de negócio, limpeza e merge
│   ├── utils/
│   │   ├── logger.py           Mecanismo de log
│   │   └── main.py              Orquestrador do Pipeline
├── requirements.txt
└── README.md
```

### O que deve conter no README.md:

1. **Explicação do Domínio:** Qual tema foi escolhido e por que o cruzamento dessas bases é importante para o negócio.
2. **Instruções de Execução:** Passos exatos para rodar o projeto (Ex: `pip install -r requirements.txt` seguido de `python src/main.py`).
3. **Dicionário de Dados:** Tabela mostrando quais colunas sobreviveram ao processo de limpeza, seus novos tipos (`string`, `int`, `date`, `float`) e uma breve descrição.

## Resultado Final Esperado:

Ao rodar o `main.py`, o pipeline deve processar os dados brutos e gerar um arquivo unificado chamado `dados_limpos_final.csv` (ou `.json`) dentro do diretório do projeto, contendo a base pronta que será exportada para o banco de dados na **Parte 2 do Projeto**.

## 5 Critérios de Avaliação

| Critério                               | Peso |
|----------------------------------------|------|
| Padrões de Projeto, POO e SRP          | 30%  |
| Qualidade da Limpeza e Lógica do Merge | 30%  |
| Tratamento de Erros e Logs             | 20%  |
| Documentação e Git                     | 20%  |